Type safety on a slice

Test 3 of docs/23: accepted programs do not go wrong, stated against D84

9 November 2026 · a paper result, with two checker probes · ochr/docs/29

Summary. Type safety holds on the slice, given three changes and two lemmas still to prove.

- **The changes.**
  - The target must say which function values may be passed: those built by accepted code (Section 1).
  - At runtime, `J` returns its transported term, which is what erasing it means (Section 4); adopted as D85.
  - The erased run must skip statements instead of running them on copies (Section 5).
- **Error reflection splits in two.**
  - *Value errors* reflect to the reference run, which an accepted definition never sends wrong. These are a constructor no arm matches, a path through a value of the wrong shape, and a call of something that is not a function. They reflect because at a closed state every value means itself.
  - *Borrow-discipline errors* (reading or matching a moved place, using an ended borrow, a dying local holding a loan of a borrow in flight, a partial content where a borrow ends or returns) reflect to the reference run too, operation by operation, using injectivity (T4), as the Lean check proves. So T4 is needed for type safety as well as consistency.
  - The exception is types with one element (`Unit`, `CellsEnd`). There a loan sits where no meaning depends on it, and the reference run is laxer than the machine (`UseDirect`). For those, borrow errors reflect to the checker's own derivation: one lemma, on borrow structure only, saying the generic run is at least as strict as the closed run.
  - D84 makes that lemma's half about moved places exact. Two probes aimed at its likeliest failure (`Unit` lenders) are rejected by the checker, as the lemma predicts.
- **The rest.**
  - Erasure holds by construction (P2, D68).
  - Recursion by name terminates by [Rec] and guarded recursion. A call through a function value needs a realizability relation (docs/18′s Q6), which brings back the old question of good ground values, but only its predicative part, since proofs are erased.
  - Well-formedness at runtime is simpler than at checking time: there are no sealed programs, so there are no inert loans.

Contents

1 The target 3
2 What this test needs from docs/24, docs/27 and D84 3
3 Error reflection 4
3.1 Every way the machine goes wrong 4
3.2 Value errors reflect to the reference run 5
3.3 Where borrow errors reflect to the reference run, and where they do not 6
3.4 Borrow errors reflect to the checker's derivation 6
4 `J` at runtime 8
5 Erasure 8
6 Termination 9
7 Well-formedness at runtime 10
8 The Lean check 10
9 What fails, and what changes 11

10 The choices 11
11 Summary for docs/18 12

1 The target

docs/18 states type safety as the second of its two targets. Consistency says proofs can be trusted; type safety says accepted programs do not crash or go wrong when run. Sharpened by this test:

Target 1 (type safety, sharpened). Let $\mathcal{P}$ be an accepted, hole-free program, d an accepted *data function* of $\mathcal{P}$ (its codomain is data or a borrow type), and $d(\overline{v})$ a *closed call*:

- every argument is a *closed value built by accepted code*: a constructor value, or a closure or global of $\mathcal{P}$, holding no abstract value and no sealed program;
- every borrow argument is a borrow of a cell holding such a value;
- every proof parameter's type, evaluated at the earlier arguments, is true in the set model.

Then the *erased run* of $d(\overline{v})$ (Definition 2):

1. terminates;
2. raises no error and never gets stuck;
3. if d returns a borrow, returns a live borrow whose content is whole;
4. leaves a result and final contents of the borrowed cells that mean what the reference run of the call says, $[\![d]\!]([\![\overline{v}]\!])$.

What changed from docs/18′s statement, and why.

- **"Built by accepted code".** docs/18 said "every parameter a concrete value". A closure value is code, and a closure from outside the program could loop or go wrong. Termination needs function arguments that behave as their meanings say (Section 6), which closures and globals of an accepted program do. The old question of good ground values asked the same ("definability") for consistency. Consistency no longer needs it, since docs/23′s assignments range over every set function; type safety does.
- **"Never gets stuck".** A closed run can stop without an error, at a `J` or a match on a proof. Section 3 lists both.
- **"True in the set model".** The proof parameters need not have closed proofs. This is the stronger statement: it covers a caller that obtains its hypotheses outside Ochr. It is also what makes a runtime `J` reachable at endpoints that are not convertible (Section 4).

Definition 2 (the erased run). The *erased run* of a closed call is the machine's run in runtime mode (RULES §3, the checker's untyped run) with three differences:

- statements (proofs, types, erased calls and their arguments) are not run, and a proof's value is $\star$;
- $\mathtt{J}(A, a, b, P, h, t)$ is t (Section 4);
- every step is D84′s: a match on a known constructor drops at its join what its other arms move; $\bot$ means moved on every path.

The arguments are closed, so the run creates no abstract value, closes nothing off, and holds no sealed program.

2 What this test needs from docs/24, docs/27 and D84

> **Reminder. docs/24 (the set model).**
> - A type means its *good* elements, so a function returning a borrow has an injective backward function.
> - A function type with borrow parameters means a function from the parameters' starting contents to the result and the cells' final contents.
> - Data is read untyped (tagged tuples), and a closure means a function on its annotated domain.
> - A recursive function means a function by guarded recursion on the rank of its decreasing argument: a call that does not decrease means the error value err. Its defining equation holds when every recursive call decreases (docs/24, the defining-equation lemma), which [Rec] ensures.

> **Reminder. docs/27 (the meaning of states).**

- A state means a function of a *future*, the final contents of its live borrows. A lent place means its final content, a borrow's holder its current content. [End] fixes the future at the current content.
- The *reference run* $\mathcal{D}[\![t]\!]$ of a term applies meanings at every call, and its [Access] ends only the borrows a read's meaning depends on.
- The checker's run of an accepted definition is below the reference run (less informative), and the reference run of an accepted definition never errs.
- The step lemmas are stated on states, whatever their origin, so they cover closed runs too.

Reminder. D84 (being implemented on branch `d84`; the prototype is branch `join-drop`).
- *Drop at the join.* A match on a known constructor runs its arm, then moves out every place of its *static join* that the arm did not move. The static join is the union of every arm's net moves, read from syntax and declared types. A stuck block takes exactly that join by move. So a closed run and the generic call leave the same places moved.
- `⊥` *means moved on every path.* It is a value of its own: `Eq` makes it equal to itself and distinct from everything else.
- `Id` *observes moves.* It also observes every owned place a side reads.
- *[Conv-bot] (D83) is retired.* `⊥` is definite, so two of them are the same information.

With D84, docs/27′s reading of `⊥` as "no information" becomes "moved", and the over-approximation by moves disappears. Moves are now exact between the generic call, a closed run and the reference run. The only steps that remain less informative are the checker's early ends (its fills tie every lender of a closed-off call).

3 Error reflection

Hardest first. The question: if the erased run goes wrong at a closed input, does something show that it cannot, for an accepted program? docs/18 proposed *error reflection*: show that the machine erring implies the reference run erring, since the reference run of an accepted definition never errs. That works for one family of errors and fails, as a lemma, for another.

3.1 Every way the machine goes wrong

`Machine.lean` on main (`a2861cbf`) has 140 sites that raise an error. Each is sorted below by whether a closed, erased run can reach it and what discharges it. Line numbers are main's; D84 moves them and adds the static join's assertion (in family I) and the join's drops, whose errors are those of [Drop] and [End] (family B).

Family	Sites (function, line)	What discharges it in a closed, erased run
V (11)	no such place (`content` 49, `setPlace` 57); not a function (`fixOf` 416, 418; `funType` 2446; `callFn` 2655); a match on a non-`Nat` value (`evalMatch` 2764); a value of another type, or a constructor no arm matches (`evalMatchInd` 3039, 3043, 3059; `evalMatchByType` 2978)	**Error reflection to the reference run** (Section 3.2). At a closed state a value means itself, so the machine's test of its shape fails exactly when the reference run's does, and that never fails for an accepted definition.
B (18)	moved out, or partly (`matchContent` 289; `evalMatch` 2756; `evalMatchByType` 2955; `evalMatchInd` 3044; `checkTailAt` 3592; `readPlace` 920, 928, 935; `borrowPlace` 962, 966; `capture` 1490, 1502); a borrow ends or returns partly moved out (`endBorrow` 734, 736; `wholeReturned` 425); a dying value borrowed by one in flight (`endLoansOfTopBind` 1015, `endLoansOfTopTemp` 1039); an argu-	**Reflection to the reference run** where the borrowed content's type has at least two elements, using (T4) injectivity (Lean, operation by operation). **Reflection to the checker's derivation** at one-element types, where the reference run is laxer (Section 3.3, Section 3.4).

	ment ended by a later one (`callFn` 2602). Under D84, also the join's drops.	
S (93)	Typing (`expectTy`, `checkFix`, `recCheck`, `splitTarget`, `checkTailAt`, `rewriteGoal`, `scrutType`, `natScrutinee`, `placeType?`, `placeType`, `projMatch`, `depFieldType`, `valType`, `fieldTele`, `indValType?`, `callType`, `declCallType`, `paramTypes`, `bindTy?`, `piSort`, `evalType`, `evalTInd`, `idType`, `repackCheck`, `lentProofAssign`, the arms of annotated matches, the proof forms `cong`, `symm`, `trans`, `rewrite`, `split`, `?`); checks on syntax at runtime positions (K2, K3, D44, D66, `&&T`, a closure moving or borrowing its captures, an unknown primitive, a constructor's arity and parameters); closing off (`wholeBorrowArgs` 435, `closeBlock`'s).	**Unreachable.** The erased run does no typing, and closes nothing off. A check on syntax at a runtime position passed when the definition was checked: the same code, in the same mode, decided from syntax and declared types (D28, D35, D55).
I (18)	assertions: the frame (`endTopLoan` 861, (L4)); the erasure pre-pass (`evalAt` 1888); D55′s sort (2019); [Close]'s precondition (2691); D82's stored types (1498, 3420); internal shapes (266, 1000, 1325, 2546, 2907, 3006, 3268, 3312, 3488, 3636, 3647); and `assignPlace`'s "overwritten while borrowed" (987), which fires only at a state with a borrow inside data (Lean: `cAssign_overLoans`). D84 adds one: the static join misses an arm's move.	**Invariants.** The frame and the pre-pass are well-formedness and syntax (Section 7). The rest concern typing or closing off, which the erased run does not do. D84′s assertion is evaluated at checking, on every arm, and its holding is what makes moves exact (Section 3.4).

Outcomes that are not errors. The machine can also *stop* without an error. A stuck closed run is going wrong too.

- **`J` at endpoints that are not convertible**, the [J-stuck] rule (2090). Removed by the runtime rule for `J` (Section 4).
- **A match with no arms outside a proof position** (D58, 2964). Reaching it means its scrutinee proves `False`. In the reference run that scrutinee means an element of $[\![\mathsf{False}]\!] = \emptyset$, so no closed run reaches it.
- **A match on a proof with several constructors, producing data** (2997). (I7) forbids it: only a subsingleton is eliminated into data, and a subsingleton has one arm.
- **A call with a neutral head, and a hole** (2649, 2164). A closed run holds no neutral, and the program is hole-free.
- **Running out of fuel.** This is termination (Section 6).

3.2 Value errors reflect to the reference run

Lemma 3 (at a closed state a value means itself). Let Ω be a state of a closed run: no abstract value, no sealed program. Then every value in Ω whose meaning is defined is a constructor value, a closure, a global, a type value, $\star$, a borrow or a loan. Its meaning under the (empty) assignment and any future is the evident one. Two values have the same constructor tag exactly when their meanings do. A value is a function exactly when its meaning is.

So a family-V test that fails at Ω because of a value's *shape* (wrong tag, missing field, not a function, wrong arity) fails on the meanings too, and the reference run's step there is err. One kind of family-V error is not visible to meanings: following * from a value that is not a borrow. A borrow means its content, so the meaning cannot tell a borrow from its content. That error is excluded by typing instead: (WF6), that a binding holds a borrow exactly when its type is a borrow type. The Lean check found the same and assumes typing in its invariant without proving it preserved (Section 8); here it is part of Lemma 4, since where borrows sit is borrow structure. The reference run of an accepted definition at arguments in its domain is never err (docs/27), and the erased run's arguments mean elements of the parameters' types. The proof parameters are true, so they mean •, which is in their type. So family V is unreachable.

Where this needs the runs to be in step. It needs the erased run's state at each step to mean what the reference run's state means at the same step. At closed states every step is exact (docs/27′s table, with no unknowns), with two exceptions:

- **[Access] at a one-element type.** The machine ends a borrow whenever its loan occurs in what is read. The reference ends it only when the read meaning depends on the borrow's final content. A loan whose content type has one element (`Unit`, `CellsEnd`, `Vec(A, Z)`) is read the same whatever the future holds, so the machine ends it and the reference does not. This changes no value (the hole had one possible content) and so no family-V test. It does change which borrows are live: that is Section 3.3.
- **`J`.** The machine's [J] is decided by conversion, which can fail at endpoints with equal meanings. The reference run's `J` is its transported term. This is Section 4.

3.3 Where borrow errors reflect to the reference run, and where they do not

The Lean check (Section 8) proves reflection to the reference run operation by operation: [End], [Access], a read, a call, and [Drop] meeting a borrower in flight. For [Drop] the runtime refuses because it sees a loan, and the reference refuses because the dropped place's meaning depends on a label held in flight. These agree because a loan always sits where the meaning depends on it. That takes backward functions to be injective (docs/18′s (T4)), together with an invariant on where holes sit (`shaped_dep`, `hole_inj`). With a constant backward function, a fill would ignore its hole while the runtime still saw the loan, and reflection would fail. So (T4) is needed for type safety too, not only for consistency's `PF` example.

The exception: types with one element. Injectivity says different final contents give different meanings. At a type with one element (`Unit`, `CellsEnd`, `Vec(A, Z)`) there are no two different contents, so injectivity is vacuous. A loan sits there although the meaning cannot depend on it. The Lean toy's contents are all numerals, so it never meets this case. A closed run of

```
def UseDirect (u : Unit) : Unit := (let r = &u; let z = u; *r := (); ())
```

ends `r` when `u` is read, since `u` holds `r`'s loan. Then `*r := ()` writes through an ended borrow, an error. A reference run with typed futures reads `u`'s meaning, `()` in every future, ends nothing, and succeeds. So at this state the machine errs and the reference run does not.

- **A loan the run placed directly** (as here) is handled by *untyped* futures, which the Lean check uses. A future may then give `r` any value, and `u`'s meaning depends on it.
- **A loan a callee's backward function placed** is not. Take a borrow of a one-element type returned through a call, as `PickU` returns one below. The reference applies the callee's meaning, whose backward function is constant on a one-element domain. It cannot tell which lender holds the hole; the run can.

The checker rejects `UseDirect` ("no such place `*r`"; probe `Scratch/UseDirectProbe.lean`), because its generic call does exactly what the closed run does. That is the general point: whether a borrow is still live is a fact about the representation, where loans sit. At one-element types the only standard as strict as the machine is the checker's own derivation.

3.4 Borrow errors reflect to the checker's derivation

Lemma 4 (the derivation is at least as strict as the run). Let d be accepted, $d(\overline{v})$ a closed call, and γ the assignment that gives d's generic unknowns the meanings of $\overline{v}$. Follow d's case tree along γ (every assignment

selects one path, docs/27). Pair each step of the erased run with the step of d's derivation that evaluates the same term at the same program point: directly where the derivation runs it, or inside the stuck block or closed-off call that contains it. At paired points:

1. *Moves agree.* A place is moved in the run exactly when it is moved in the derivation. (D84.)
2. *Ends are fewer.* Every borrow the run has ended has been ended in the derivation, or is inside a block or closed-off call the derivation treats as one step.
3. *Ties are fewer.* Every position of the run holding a live loan has, in the derivation, a corresponding position holding a loan of the corresponding borrow, or a fill holding its hole.

So every family-B error of the run has a counterpart in the derivation at or before the paired point, and d would have been rejected. The lemma is needed only for borrows whose content type has one element, where reflection to the reference fails (Section 3.3). Everywhere else reflection to the reference suffices.

Proof sketch. By induction on the run.

- **Steps that only read syntax and labels** ([Borrow], [Read], [Move], [Copy], [Assign], [Drop], [End], a call's push and pop) do the same thing to the borrow structure on both sides. Their decisions are read from syntax and declared types: copy or move (D80), the class (D77), the [Close] row (D48). They do not depend on whether a value is concrete or abstract. [Access] ends the borrows whose loans occur in what is read. By clause 3, the derivation's state has at least those loans, so it ends at least those borrows (clause 2).
- **A match on data.** Where the derivation splits (tail position), γ selects the arm the run takes (coverage), and D84 drops the same static join after it on both sides (`joinCont`). Where the derivation closes the match off as a block, the block takes the static join by move (clause 1) and the places its arms write or borrow by `&`. Its fills tie at least the lenders the arm ties (clause 3). D84′s assertion, checked on every arm at checking time, guarantees that no arm moves outside the static join.
- **A call the derivation closes off and the run runs.** For a borrow-free result, the lenders get fills without loans in the derivation and final contents without loans in the run. For a result $\&T$, the derivation's fills hold the hole in *every* lender (no η at `Unit` in this row), and the run's only in those the callee actually returned a part of. Clause 3 holds, and clause 2 follows: what the run ends on reading a lender, the derivation ends too.
- **A call both run.** The callee's run, by induction on program position (the callee is an earlier accepted definition, or d at a smaller argument).

□

Probes of its likeliest failure. D80′s R3 makes a closed-off call's fill of a lent `Unit` place `()`, with no loan. If that applied to the row for a borrow result, the derivation would hold no loan where the run holds one. Clause 3 would fail, and a `Unit` lender read after the call would end the returned borrow in the run but not in the derivation. Two programs aimed at it (`Scratch/UnitHoleProbe.lean`, scratch worktree, uncommitted):

- `UseU`: `let r = PickU(n, &u1, &u2); let z = u1; *r := ()`, with `PickU (n : Nat) (x y : &Unit) : &Unit`;
- `LeakU`: `let u = (); PickU(n, x, &u)`, which returns, at $n = 1$, a borrow of its own local.

Both are rejected on main ("no such place `*r`"; "[Drop] u dies while a value in flight borrows it"). In the row for a borrow result, `closeCall` puts the hole in every fill and applies R3 only in the borrow-free row (`Machine.lean`, D80 R3). So the derivation ties `Unit` lenders too, as the lemma needs.

Status. A paper argument, sketched by cases; no counterexample. It is the one place type safety compares two runs, and it is needed only at one-element types. It does so on borrow structure only (labels, loans, moved places), never on values, which come from meanings (Section 3.2). Before D84 its first clause was an inequality (a block could move what the selected arm keeps). D84 makes it an equality.

Adequacy and liveness follow. With no error, the erased run's result and final contents are computed by exact steps and early ends. An early end fixes a hole at the borrow's current content, which the run never changes afterwards, since it never uses an ended borrow. So the final meanings are the reference run's (clause 4 of Target 1). A returned borrow is live: the run's final value is a borrow, not $\bot$; `wholeReturned` checks its content whole; and D65 refuses a borrow of a dying local (clause 3).

4 J at runtime

The machine's [J] computes $\mathsf{J}(A, a, b, P, h, t)$ to t only when a and b are convertible, and is stuck otherwise. Conversion is incomplete for functions: two closures with equal meanings and different code are not convertible. So a closed run can reach a `J` whose endpoints have equal meanings but is stuck there.

A program that reaches it.

```
inductive Box (k : Π(n : Nat). Nat) := MkBox(x : Nat)
def CastFn (f g : Π(n : Nat). Nat) (h : Eq(Π(n : Nat). Nat, f, g)) (b : Box(f)) : Box(g) :=
  J(Π(n : Nat). Nat, f, g, λ(k : Π(n : Nat). Nat). Box(k), h, b)
```

Call it with `f = λ(n : Nat) : Nat => n` and `g = λ(n : Nat) : Nat => match n { Z => Z, S m => S(m) }`.

- Both mean the identity on $\mathbb{N}$, so h's type is true, and the call satisfies Target 1.
- Their generic observations are σ and the sealed $\lceil g(\sigma) \rceil$, which are not convertible.
- So the machine's run stops at `J`.

The motive returns a data type, so this `J` is not a proof and the erased program does run it. The Lean check makes the same point on its toy machine (`cast_fn_counterexample`): the checker's rule is stuck at runtime on $n + 0$ against $0 + n$, one function in two encodings. First-order endpoints with equal meanings are equal after normalisation, so they do not arise (`cast_counterexample` shows the case normalisation removes).

Can a closed program reach it? Only if a closed proof of $\mathsf{Eq}(\Pi(n : \mathsf{Nat}).\ \mathsf{Nat}, f, g)$ exists for non-convertible f and g. `refl` proves only convertible equations. A lemma proving the equation at its generic call, where the two sides were convertible, would need them to stop being convertible at a closed argument. That would break docs/18′s (V9), that conversion survives refinement, which no one has seen fail. So probably not. But Target 1 quantifies over proof parameters that are true, not proved, so it does reach it.

Adopted (D85, lane `runtime-j`): at runtime `J` is its transported term. In the erased program $\mathsf{J}(A, a, b, P, h, t)$ is t. The checker keeps [J] and [J-stuck] for typing, and for its own untyped runs.

- **It is what erasure means.** A, a, b, P and h are statements (type slots, endpoints, motive, proof). The erased program does not have them, so it cannot compare the endpoints. `J` is a cast, and an erased cast is the identity. D85 records it as the erased runtime's rule.
- **It is sound given membership** (Lean: `castRT_safe`). The run is at a closed call whose proof parameters are true, and the reference run never errs. So at every `J` the run reaches, h means an element of $[\![\mathsf{Eq}(A, a, b)]\!]$, and a and b have equal meanings. Then $P(a)$ and $P(b)$ have equal meanings, and t, a value of $P(a)$, is a value of $P(b)$. Without true proof parameters it would not be sound: `J(Nat, 0, 1, λz. Vec(z), h, v)` under a false `h : Eq(Nat, 0, 1)` would return a `Vec(0)` as a `Vec(1)`. That is why the target requires them.
- **It does not break "decisions never read a run".** At runtime the rule is unconditional: it reads nothing. Whether a position is at runtime or in a statement is decided from syntax (P2). The checker's [J], which reads conversion, stays where it is, value-dependent by design (census site 18).
- **It needs the modes lemma, in one clause.** The erased run and the machine's untyped run differ at `J`. Where the machine computes `J`, both give t. Where the machine is stuck, the erased run gives t, and so does the meaning (docs/24, choice C9: `J` means its transported term). So the erased run is the more defined of the two and agrees with the meaning everywhere (Section 5).

5 Erasure

The machine runs every statement on a private copy that it then discards (D68). The erased program does not run them at all. Beyond the first-order fragment, the two give the same result and effects, for four reasons.

1. **A statement's run leaves no trace.** Its copy is discarded, its moves included (D68 (E5)). So skipping it leaves the same state.
2. **No runtime step reads a statement's value except as $\star$.** Each case is decided by syntax:
   - A proof is $\star$ wherever its declared sort is `Prop` (D55).

- Constructor values record their parameters, but no step reads them (V2).
- A match on a proof is decided by declared types (D45). Producing data, it is on a subsingleton, which has one arm (I7). Producing a proof, it is erased itself. The runtime path in `evalMatchByType` runs the one arm and never inspects the proof.
- A call's class, and so whether it is erased, is read from its Π-type's syntax (D54, D77).
- Type values never sit in data: a field of type $\texttt{Type}_0$ would live in $\texttt{Type}_1$, which (I6) forbids.

3. **Calls through function values erase alike.**
   - A call does not consume its function (D53), in both runs.
   - A closure that may not move its captures (D53) is checked when the closure is checked, on the same syntax.
   - A closure whose body is a statement (one returning a type or a proof) is erased at its call, by its class, in both runs.
   - A captured proof is inlined as `(⋆ : T)`, which is erased.
4. `J` is the clause of Section 4.

Lemma 5 (erasure). For a closed call of an accepted data function, if the machine's untyped run completes, the erased run completes with the same result and the same final state on every non-statement position. If the machine's run is stuck at a `J`, the erased run continues with its transported term. The machine's run of a statement can fail or run out of fuel; the erased run never runs it.

So the erased run is at least as defined as the machine's, and Target 1 is stated for the erased run.

6 Termination

Recursion by name terminates. Every recursive call is by name ([Rec], D78), at a strict subterm of the entry value as refined. At a closed call the entry value is concrete, and the argument the run passes is that subterm: a component, so it has smaller rank. By docs/24′s lemma on the defining equation, the guard never fires and the function's meaning satisfies its equation. The runs of recursive calls form a chain decreasing in (program position, rank of the decreasing argument).

Calls through function values are not covered by that measure. A call `x(u)` of a closure runs the closure's body. That body is part of the program, but it is not smaller than the call site in any order the run's measure sees. A closure passed into a function, or stored in data through a type parameter (`List(Π(n : Nat). Nat)`), can be called from anywhere. Nor does the set model alone give termination. A function's meaning is its graph, and nothing about graphs bounds the steps a machine takes to compute a value. Relating the two is *adequacy*, which needs a logical relation.

Definition 6 (realizers (to be built)). By the same recursion as docs/24′s model (program position, the structure of the type term, the rank of a decreasing argument), give each type A a relation $v \underset{A}{\vDash} e$ between closed runtime values v and elements $e \in [\![A]\!]$:

- at data: v means e, and its function-valued components (through type parameters) realize their meanings;
- at a proposition or a sort: always (they are erased);
- at a Π-type of class "other": for every argument realizing its parameter's type (a cell holding one, for a borrow parameter), the erased run of the call terminates without error. Its result realizes the result's meaning, and the cells' final contents realize theirs. For a borrow result, writing a realizer through the returned borrow and ending it leaves cells realizing the backward function's value;
- at a type variable $A : \texttt{Type}_i$: as the type it stands for, which needs $[\![\texttt{Type}_i]\!]$ to be read as *pairs* of a set and a realizer relation (Girard's candidates). A Π over $\texttt{Type}_i$ lives in $\texttt{Type}_{i+1}$, so the definition is predicative and well founded by level.

Lemma 7 (realizability (fundamental lemma, to be proved)). Every accepted definition's value realizes its meaning; every closure built by an accepted run realizes its meaning.

With it, every call in a closed run terminates. A recursive call does so by induction on rank. A call of an earlier global, a closure, or a function parameter does so by Lemma 7. That is why Target 1 asks for arguments built by accepted code: they realize their meanings, and arbitrary closures need not.

Status. Recursion by name: paper argument, from [Rec] and docs/24. Higher-order calls: reduced to Lemma 7, which is the classic Tait–Girard argument and is not done. It is docs/18′s Q6 (realizability for calls through function values): the old question of good ground values, returning only for type safety and only in its predicative part. Runtime erases proofs, so impredicative `Prop` never enters the relation. Types computed by code (`Cells(E, n)`) are handled as docs/24 handles them, by recursion on terms, not on types. docs/24 showed the model needs no pairs. Realizability does, at universes: the pairs return here, but as a relation on runtime values, not as a second notion of "good".

7 Well-formedness at runtime

(WF1)–(WF7) of docs/18 are preserved by every step of a closed run. This is shared with consistency. A closed run is the easier case: it holds no sealed program, so there are no inert loans, and (WF2)'s exception for them is vacuous.

- **(WF1), one borrow per label.** [Borrow] takes a fresh label, and moving a borrow moves its one holder.
- **(WF2), every loan's borrow is in the state.** A borrow leaves the state only by [End], which substitutes for every loan of its label. That happens at [Drop] of a value holding it, at an overwrite (`assignPlace` ends the old borrow), at a frame's pop, and at D84′s join (`dropAtJoin` ends a borrow it drops). Each needs (WF3) to terminate and to give one result.
- **(WF3), no cycles among borrows.** A new borrow's content comes from a place [Access] has just cleared of loans. A reborrow's content is a part of its parent's, which cannot hold the parent's own loan. [End] composes edges, which keeps a relation acyclic.
- **(WF4), closures capture no borrow.** `capture` refuses a borrow, statically.
- **(WF5), arguments exclusive at a call.** [Access] runs before each argument is evaluated, and a later argument that ends an earlier one's borrow is an error (`callFn` 2602, family B). The Lean check found (WF5) is about causality: with a live loan in an argument, a read now would depend on a write later.
- **(WF6), values of their types.** Membership: at a closed state, the reference run's values are in their types (Section 3.2).
- **(WF7), [Rec] and `&` only at `Type₀`.** Static.

The frame property is the lemma that a call's run touches only its own frame and the places its borrow arguments lent ((L1)–(L5)). It makes a call's effect at runtime its isolated run, plugged back into the caller. It follows from (WF1)–(WF5), with (L4) asserted by `endTopLoan`.

Status. paper argument, sketched per step; shared with consistency. The Lean check proves (WF2), (WF3) and (WFc) preserved by [End], [Access], [Drop]'s loop and a call (Section 8). Its (WF3) is that a borrow's content holds only loans of later labels, and (WFc) that no borrow sits inside a borrow's content. (WF2) needs (WF3) twice. It does not cover [Borrow], assignment and moves at field paths, or (WF1).

8 The Lean check

`Ochr/Model/Safety.lean` (commits `0c86c1ed`, `24edd422`, `7568feb3`, `755378ca`), standalone, Lean core only. It builds with no `sorry`; its axioms are `propext` and `Quot.sound`. It is a *concrete runtime* on docs/27′s toy machine: no unknowns, and σ_g is a concrete function with an injective backward function. A call computes its forward value at once and fills the lent place with a pending backward computation over the hole, which runs when [End] fills the hole (`normV`; `ev_normV` keeps the meaning). Errors are explicit.

Error reflection, operation by operation (Section 3):

- "not whole" means no information somewhere, under every future (`notWhole_hasBot`);

- what the runtime finds at a place is its meaning, and an ended borrow or a moved part means no information (`cFollow_ok`, `cFollow_err`);
- an end refuses exactly where the holder means no information (`cEndL_ok`, `cEndL_err`);
- [Access] loses only information (`cAccess_ok`, `cAccess_err`);
- a read's errors are running out of fuel, an ill-typed path, or an error of the reference run (`cClone_reflect`);
- assigning over a borrowed place fires only at a state with a borrow inside data (`cAssign_overLoans`);
- a successful call means the reference call (`cCall_ok`, `cCall_err`);
- [Drop] meeting a borrower in flight reflects only because meanings are injective in their loans (`cDropLoop_inFlight`, `shaped_dep`, `hole_inj`), so (T4) is needed for type safety.

Well-formedness (Section 7): `cEndL_wf`, `cAccess_wf`, `cDropLoop_wf`, `cCall_wf`.

Runtime `J` (Section 4): `castRT_safe`, `cast_counterexample`, `cast_fn_counterexample`.

D84: `join_armZ`, `join_armS` and `join_meaning`: with the drop at the join, both arms leave the moved places $\bot$, exactly the block's reading, so $\bot$ means "definitely moved". `nojoin_armS` and `nojoin_meaning`: without it, the block is only below the run.

Its gaps, which this document shares or addresses:

- Following * from a non-borrow is a representation error meanings cannot see. Reflection needs typing in the invariant, which the check assumes and does not show preserved (Section 3.2).
- Its shape invariant is a hypothesis; preserving it needs hole labels to be numeral-typed. That is exactly the one-element-type exception of Section 3.3.
- Reflection is per operation, not a theorem over a whole run.
- Termination is not proved; it uses fuel (Section 6).

9 What fails, and what changes

- **No checker bug found.** The likeliest failure of Lemma 4, η at `Unit` in a closed-off call's fills, does not occur: R3 applies only to the borrow-free row (probes `UseU` and `LeakU`, both rejected).
- **A rule change, at runtime only: `J` is its transported term** (Section 4), adopted as D85. The checker is unchanged.
- **A needed sharpening of the target.** Function arguments must be built by accepted code (Section 1). Without it termination is false: a closure from outside the program can loop.
- **A weaker form of error reflection.** "The machine errs implies the reference run errs" is false as a lemma at one-element types (`UseDirect`). Elsewhere it holds, given (T4) injectivity (Lean, per operation). At one-element types, borrow errors reflect to the checker's derivation (Lemma 4), the one comparison of two runs that type safety needs.
- **(T4) is needed for type safety**, not only for consistency: [Drop]'s reflection rests on meanings being injective in their loans.
- **Two lemmas to prove:** Lemma 4 (paper sketch) and Lemma 7 (not done).

10 The choices

- **T1. Type safety is stated for the erased run**, not the machine's untyped run. The erased run skips statements and treats `J` as its transported term (Definition 2).
- **T2. Proof parameters are true, not proved.** This is the stronger target, and it is why runtime `J` must not decide by conversion.
- **T3. Function arguments are built by accepted code.** They are realizers (Definition 6).
- **T4. Borrow errors are discharged by the derivation, value errors by the reference run** (Section 3).
- **T5. Everything is stated against D84.** It makes moves exact, so Lemma 4′s first clause is an equality.

11 Summary for docs/18

1. **Type safety is sharpened.** The erased run, which skips statements and makes `J` its transported term, terminates, never errs or sticks, returns live borrows, and means what the reference run says, at every closed call whose function arguments are built by accepted code and whose proof parameters are true.
2. **Error reflection splits.** The machine's 140 error sites fall into four families. Value errors reflect to the reference run, since at a closed state values mean themselves. Borrow errors reflect to it too, operation by operation, using (T4) injectivity (Lean), except at one-element types, where a loan sits that no meaning depends on. There they reflect to the checker's derivation, which is at least as strict as the closed run on borrow structure (Lemma 4). Static and typing errors are unreachable, and internal assertions are invariants.
3. **At runtime `J` returns its transported term** (D85, docs/18′s Q5), which is what erasure means. It is sound by membership, reads nothing, and is the one clause erasure adds to the modes lemma. Erasure otherwise holds by construction (P2, D55, D68).
4. **Termination.** Recursion by name terminates by [Rec] and guarded recursion. Calls through function values need a realizability relation (docs/18′s Q6), for type safety only, and predicative because proofs are erased. Not done.
5. **Well-formedness at runtime** is (WF1)–(WF7), with no inert loans. D84 makes moves exact between the generic call and the closed run. Probes of the likeliest gap (`Unit` lenders) are rejected as predicted.